DEVPILOT AI

AI Architecture & LLM Routing

Specification

Defines the provider-independent, free-tier-aware AI layer of DevPilot AI: the AI Gateway, provider routing, failover, privacy policy, token optimization and the AI flows used across all modules.

Document Version	1.0
Project	DevPilot AI
Backend	ASP.NET Core Web API
Database	SQL Server
Backend Hosting	MonsterASP
Frontend	Undefined yet
Frontend Hosting	Cloudflare Pages
Email	Brevo

Contents

1. Purpose 4
2. Core AI Architecture 5
3. Important Design Decision 6
4. Provider Types 7
5. Current Free-API Sources (Provider Catalog) 8
6. Recommended Initial Provider Pool 9
7. Google Gemini 10
Suggested Uses 10
Privacy Consideration 10
8. Groq 11
Suitable Use Cases 11
9. OpenRouter 12
Advantages 12
Important 12
10. CodeCraft API 13
Role in DevPilot AI 13
11. Provider Configuration Model 14
12. AI Request Routing 15
13. Provider Selection Rules 16
Example: Debugging with an Image Screenshot 16
14. Task-Based Model Routing 17
15. Token Optimization Pipeline 18
16. Similarity + Solution Reuse 19
17. Solution Synthesis 20
18. AI Response Contract 21
Debugging / Solution 21
Code Review 21
Challenge Evaluation 22
19. Response Validation 23
20. Retry / Failover Policy 24
21. Do Not Randomly Rotate Providers 25
22. Provider Cooldown 26
23. Provider Health Monitoring 27
Possible States 27
24. AI Request Caching 28

25. Prompt Management 29
26. Private Code / Data Protection 30
27. GitHub Repository Context Optimization 31
Possible Context Sources 31
28. Image / Screenshot Debugging 32
29. Telegram AI Flow 33
30. Article Generation Flow 34
31. Challenge Generation Flow 35
32. Challenge Evaluation Flow 36
33. Code Review Flow 37
34. AI Usage Tracking 38
35. Free-Tier Safety Rule 39
36. Admin AI Dashboard 40
37. Recommended Initial Routing Strategy 41
38. Development Implementation Order 42
39. Final AI Architecture 43
40. Final Design Goal 44

1. Purpose

This document defines the AI architecture for DevPilot AI. The AI layer is responsible for:

- Developer problem debugging.
- Root-cause explanation.
- Step-by-step solutions.
- Solution adaptation.
- Educational article generation.
- Article improvement.
- Challenge generation.
- Challenge evaluation.
- GitHub repository analysis.
- AI code review.
- Solution synthesis.
- Telegram AI fallback.

> **Core requirement**
> The system must not depend on one LLM provider. Instead, DevPilot AI uses an internal AI Gateway that can route requests across multiple free/free-tier providers and models.

2. Core AI Architecture

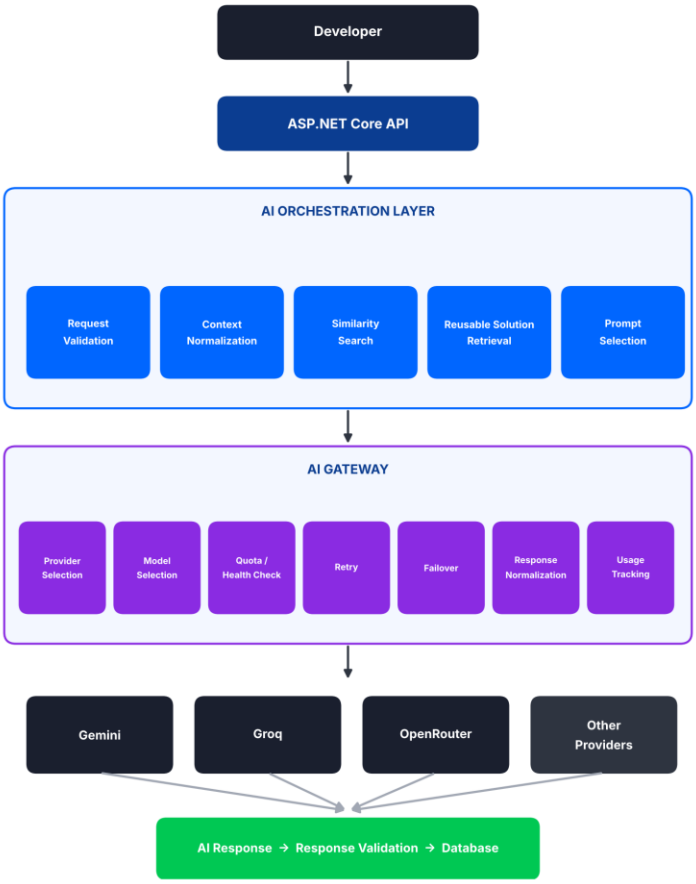


Figure 1 – Core AI architecture: from developer request to stored response

3. Important Design Decision

The application must never call a provider directly from business services. All AI traffic flows through the AI Gateway and its Provider Router, which prevents the application from becoming dependent on any one provider.



Figure 2 – Direct provider calls (incorrect) versus routing through the AI Gateway (correct)

4. Provider Types

The AI Gateway should support two provider categories, and the architecture treats both as AI providers.

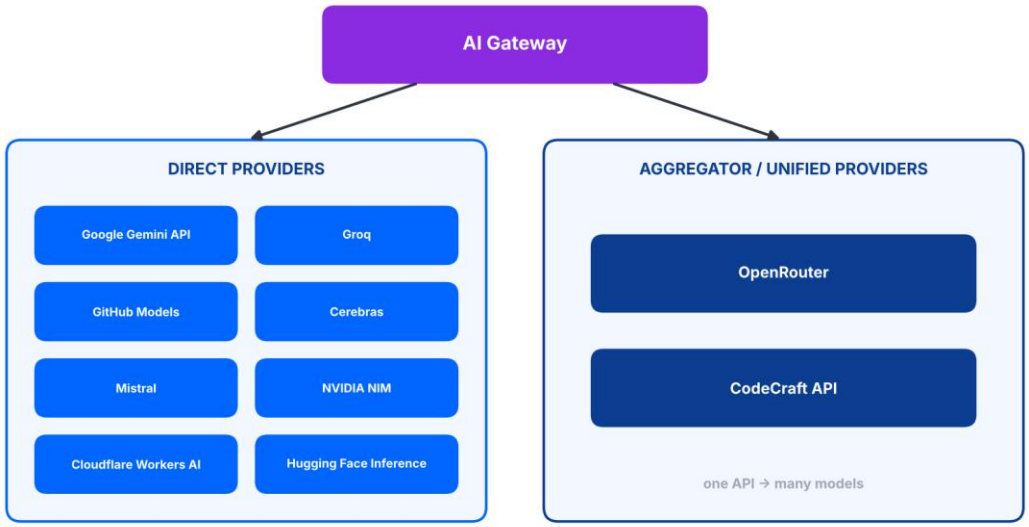


Figure 3 – Direct providers and aggregator / unified providers

Category	Description	Examples to evaluate
4.1 Direct Providers	The application communicates directly with the provider API.	Google Gemini API, Groq, GitHub Models, Cerebras, Mistral, NVIDIA NIM, Cloudflare Workers AI, Hugging Face Inference Providers
4.2 Aggregator / Unified Providers	One API may expose multiple models.	OpenRouter, CodeCraft API

5. Current Free-API Sources (Provider Catalog)

The project team will use these sources as discovery/catalog resources:

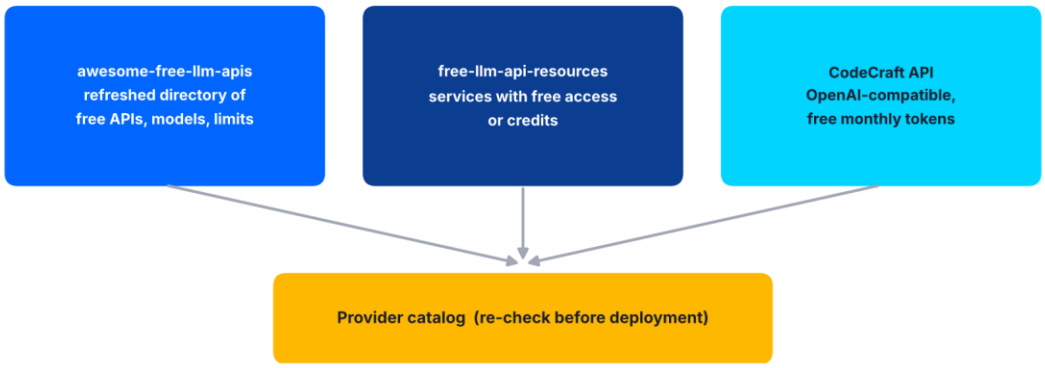


Figure 4 – Catalog sources feeding the provider catalog

Source	Description
A. awesome-free-llm-apis	Maintains a frequently refreshed directory of free LLM APIs, providers, models, rate limits and related configuration information.
B. free-llm-api-resources	Lists services providing free access or credits for API-based LLM usage and currently includes providers such as OpenRouter, Google AI Studio, NVIDIA NIM, Mistral, Hugging Face, Cerebras, Groq, Cohere, GitHub Models and Cloudflare Workers AI.
C. CodeCraft API	Provides an OpenAI-compatible endpoint and currently advertises a free tier with a monthly token allowance.

Re-check before deployment

The team must re-check the provider catalogs before deployment because free plans, limits and model availability can change.

6. Recommended Initial Provider Pool

Recommended candidates for the first implementation:

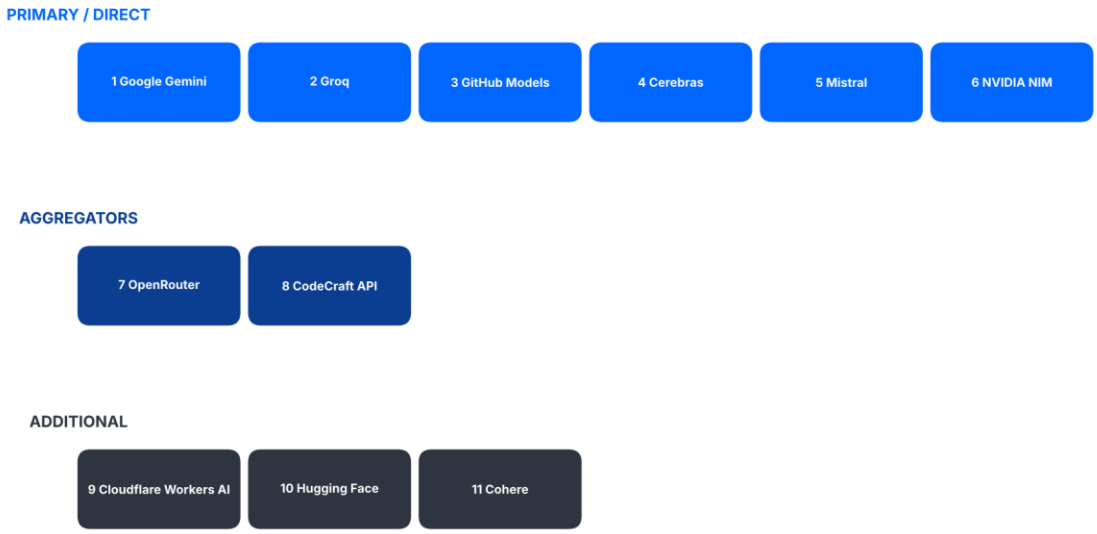


Figure 5 – Recommended provider pool by tier

> **Configuration, not code**
> The exact active model list must be stored in configuration/database and must not be hard-coded permanently.

7. Google Gemini

Google Gemini should be included as a major provider. Current Google documentation provides a free developer tier with free input and output tokens for eligible models and access through Google AI Studio. The current pricing page also lists multiple Flash-family models with free-of-charge standard input/output pricing.

Suggested Uses

- Debugging.
- Root-cause analysis.
- Article generation.
- Code review.
- Repository analysis when the context is within model limits.

Privacy Consideration

> **Free-tier data use**
> Google's current pricing documentation states that content in the free tier is used to improve Google's products.

Therefore:

- Do not silently send sensitive/private code to the free Gemini tier.
- Provider privacy policy must be part of provider configuration.
- Private-code routing must be controlled by policy.

8. Groq

Groq is a strong candidate for fast coding/debugging requests. Current Groq documentation exposes rate-limit dimensions, and Groq also returns rate-limit headers that the AI Gateway can use to understand remaining request/token capacity and reset timing. The AI Gateway should therefore prefer runtime quota headers over hard-coded numbers.

Dimension	Meaning
RPM	Requests per minute
RPD	Requests per day
TPM	Tokens per minute
TPD	Tokens per day

Suitable Use Cases

- Short debugging.
- Fast explanations.
- Code review.
- Interactive developer conversations.

9. OpenRouter

OpenRouter should be treated as a provider/aggregator. Its free-model collection currently contains multiple $0 models, and the OpenRouter free-model router can select from available free models.

Advantages

- Many models behind one API.
- Useful fallback when a direct provider is unavailable.
- OpenAI-compatible API shape.

Important

- OpenRouter free models do not all have identical privacy/retention terms.
- The provider/model metadata must store privacy and usage policies.
- A free model may disappear or move away from the free tier.

10. CodeCraft API

CodeCraft API provides an OpenAI-compatible endpoint. Its current site advertises:

- Free monthly token allowance.
- Multiple models.
- Streaming.
- Tool calling.
- Vision input for capable models.
- Usage analytics.

Role in DevPilot AI

- A fallback provider.
- A specialized provider for model capabilities.
- A compatibility-friendly provider because of its OpenAI-style API.

> **Limited quota**
> The free tier should be treated as limited quota, not unlimited free usage.

11. Provider Configuration Model

Each provider should have configuration stored as two related entities: AIProvider and AIModel (one provider has many models).

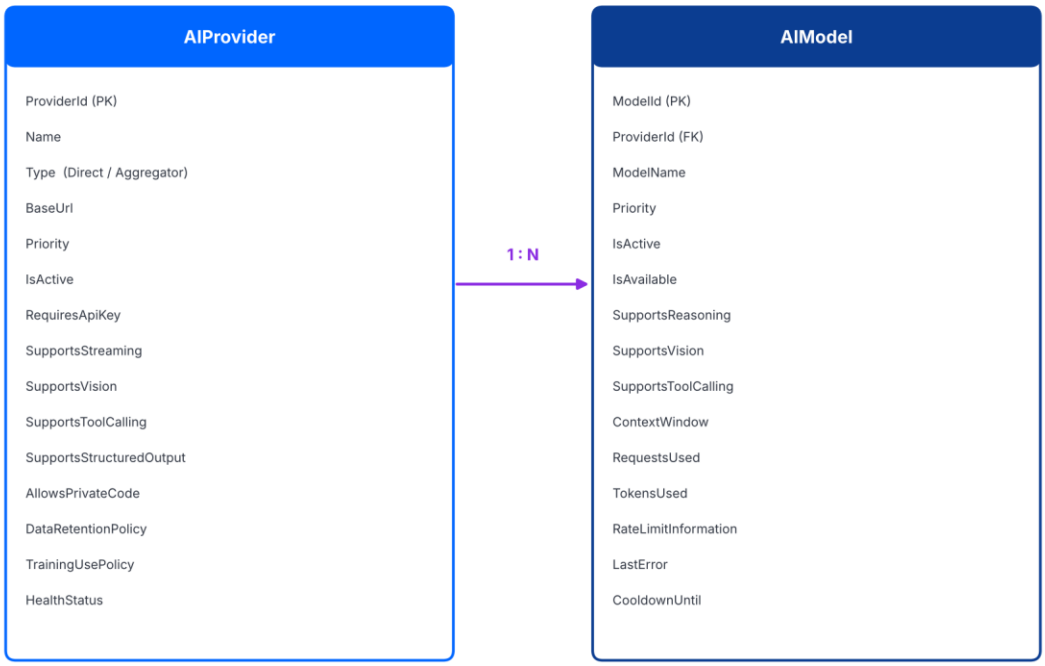


Figure 6 – AIProvider and AIModel configuration entities

12. AI Request Routing

Every AI request follows this twelve-step process:

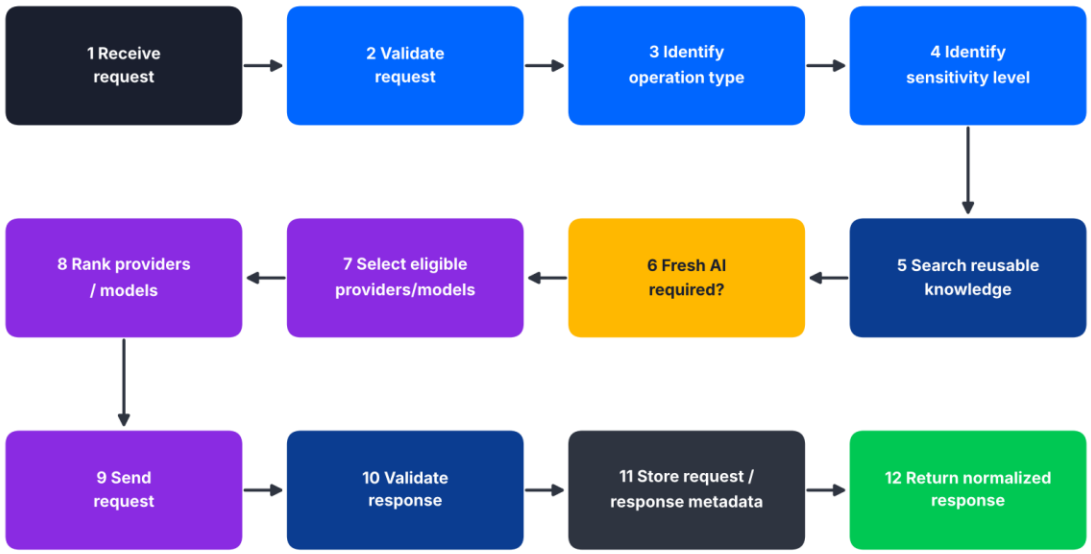


Figure 7 – AI request routing process

13. Provider Selection Rules

Provider selection should consider:

1. Provider active status.
2. Model active status.
3. Current availability.
4. Rate-limit/usage availability.
5. Required capabilities.
6. Data/privacy policy.
7. Task type.
8. Context size.
9. Provider priority.
10. Recent failure rate.

Example: Debugging with an Image Screenshot

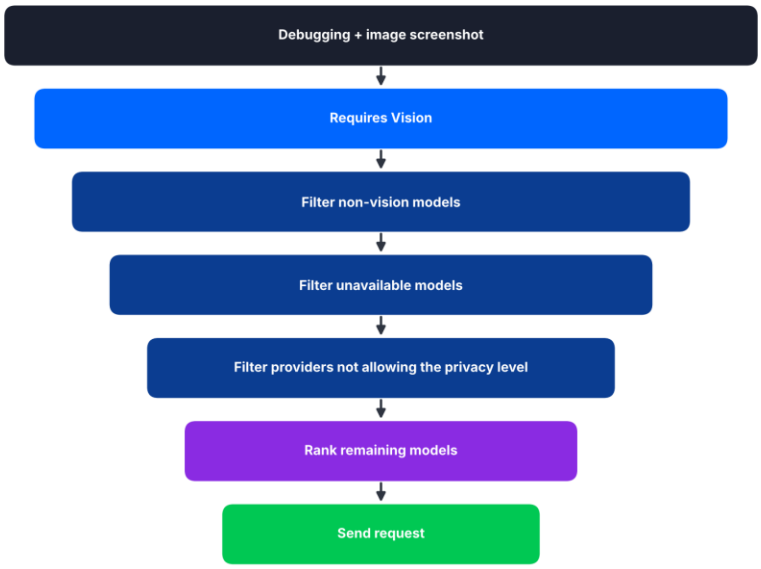


Figure 8 – Eligibility filtering and ranking for a vision request

14. Task-Based Model Routing

Different operations can prefer different model characteristics.

#	Operation	Preferred model characteristics
14.1	Debugging	Strong coding · Strong reasoning · Good context capacity
14.2	Root Cause Explanation	Strong reasoning · Good instruction following
14.3	Article Generation	Good writing · Strong technical accuracy · Large context
14.4	Challenge Generation	Strong coding · Good structured output
14.5	Challenge Evaluation	Strong code reasoning · Ability to follow evaluation criteria
14.6	Code Review	Strong coding · Long context · Good structured output
14.7	Repository Analysis	Large context · Strong coding/reasoning · Tool support when required
14.8	Simple Rewriting / Adaptation	Fast and efficient model

15. Token Optimization Pipeline

The AI Gateway must not be the first step. Normalization and similarity search come first; a suitable known solution is retrieved and a smaller context is built, and fresh AI is used only when no suitable solution exists.



Figure 9 – Token optimization pipeline

> **Result**
> This reduces unnecessary token usage.

16. Similarity + Solution Reuse

For every new problem, the system searches:

- Problem description
- Error message
- Stack trace
- Technology
- Framework
- Relevant code
- Existing embeddings

If a sufficiently similar problem is found:

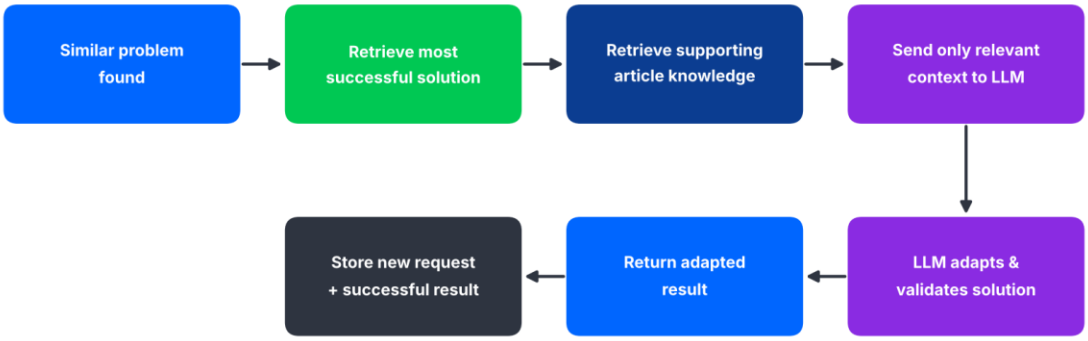


Figure 10 – Solution reuse flow

1. Retrieve the most successful solution.
2. Retrieve supporting article knowledge.
3. Send only relevant context to the LLM.
4. Ask the LLM to adapt and validate the old solution.
5. Return the adapted result.
6. Store the new request and the successful result.

> **Rule**
> Do not blindly return historical answers.

17. Solution Synthesis

If multiple successful solution variants exist, the system may send the evidence to the AI Gateway. The AI produces an improved solution, which becomes a new knowledge article version. The previous versions remain stored.

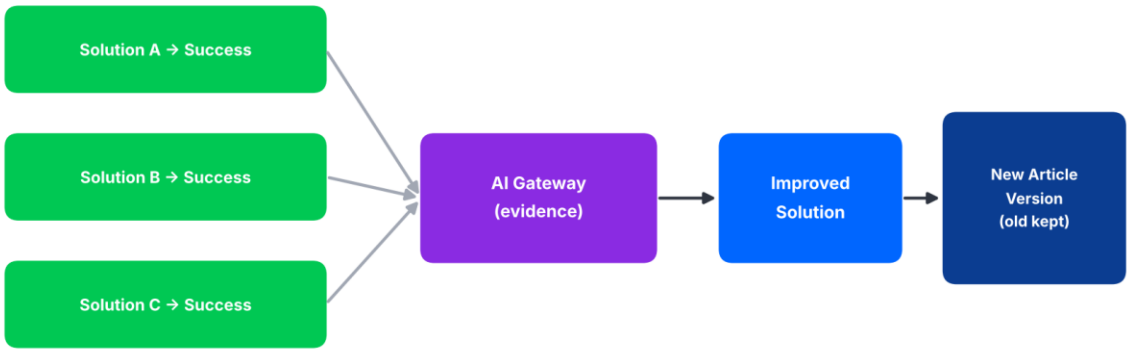


Figure 11 – Solution synthesis into a new article version

18. AI Response Contract

AI responses should be normalized into a predictable internal structure. Three structures are defined: debugging/solution, code review, and challenge evaluation.

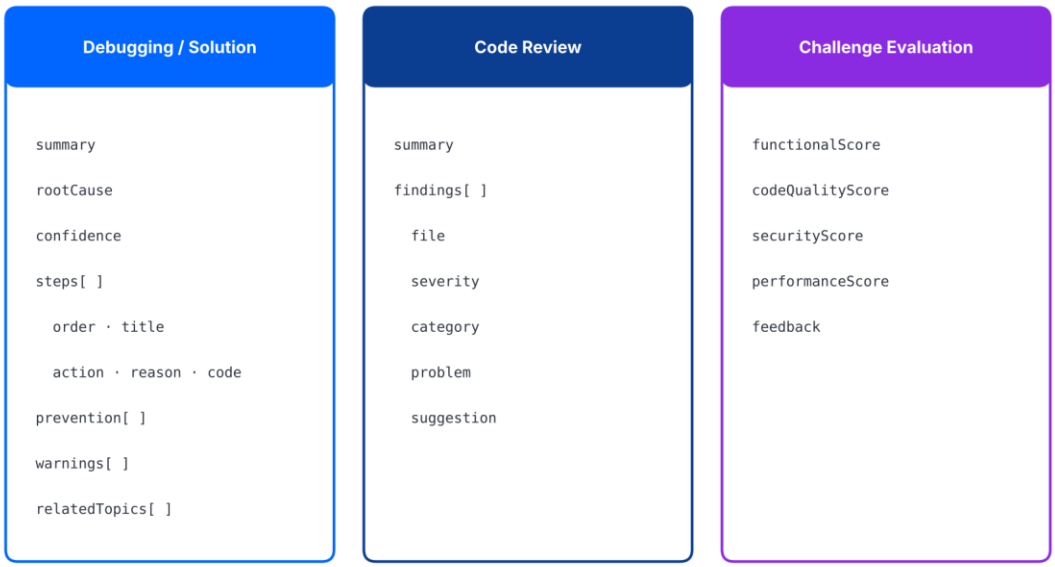


Figure 12 – Normalized AI response structures

Debugging / Solution

```
{
  "summary": "...",
  "rootCause": "...",
  "confidence": 0.0,
  "steps": [
    {
      "order": 1,
      "title": "...",
      "action": "...",
      "reason": "...",
      "code": "..."
    }
  ],
  "prevention": [],
  "warnings": [],
  "relatedTopics": []
}
```

Code Review

```
{
  "summary": "...",
  "findings": [
    {
      "file": "...",
```

```
      "severity": "...",
      "category": "...",
      "problem": "...",
      "suggestion": "..."
    }
  ]
}
```

Challenge Evaluation

```
{
  "functionalScore": 0,
  "codeQualityScore": 0,
  "securityScore": 0,
  "performanceScore": 0,
  "feedback": "..."
}
```

19. Response Validation

The API should not trust raw LLM output. It validates required fields, JSON structure, maximum sizes, allowed enum values, safe content formatting, unexpected output and provider-specific response format.

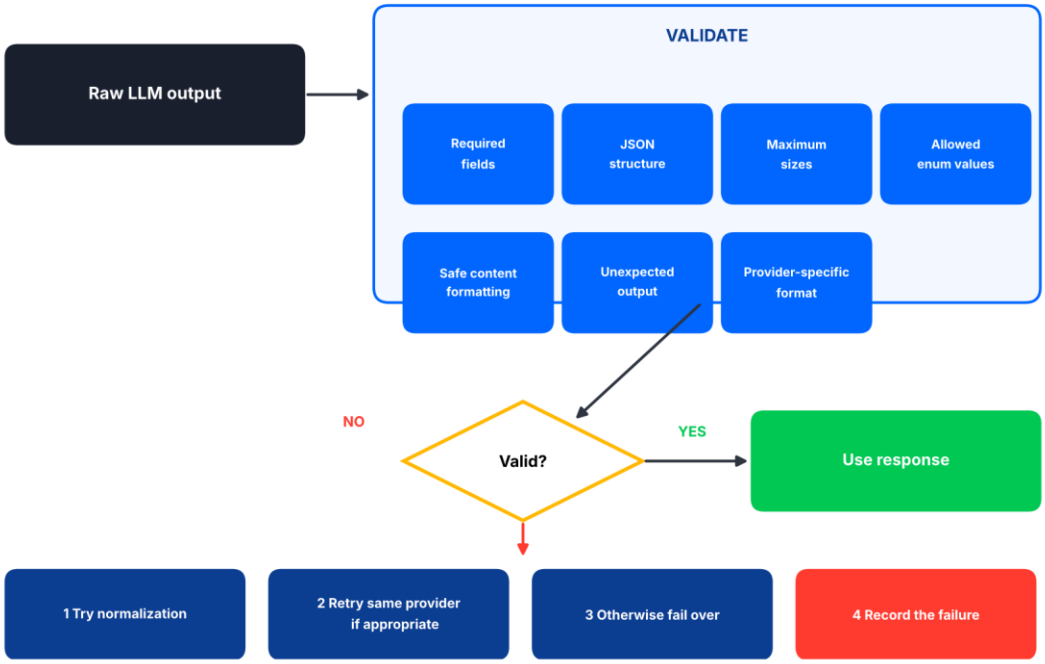


Figure 13 – Response validation and invalid-response handling

If the response is invalid:

1. Try response normalization.
2. Retry the same provider if appropriate.
3. Otherwise fail over.
4. Record the failure.

20. Retry / Failover Policy

Failures are grouped into five categories, each with a defined action.

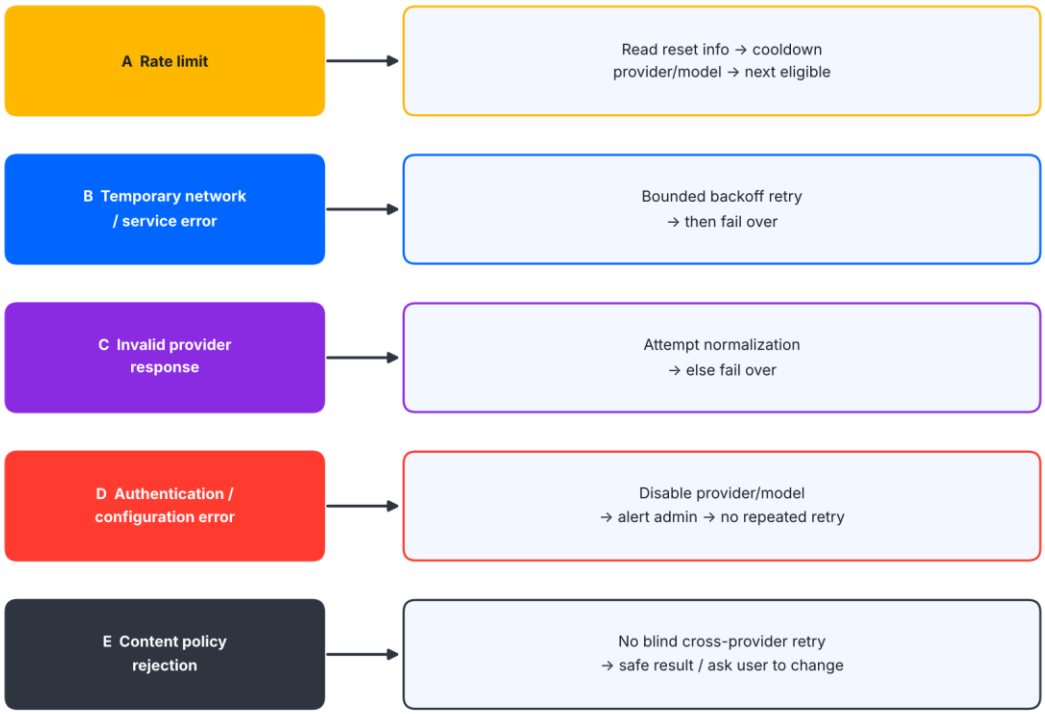


Figure 14 – Failure categories and actions

Failure category	Action
A. Rate limit	Read provider reset information when available. Put provider/model into cooldown. Select next eligible provider.
B. Temporary network/service error	Retry with bounded backoff. Then fail over.
C. Invalid provider response	Attempt normalization. If unsuccessful, fail over.
D. Authentication/configuration error	Disable provider/model. Alert administrator. Do not repeatedly retry.
E. Content policy rejection	Do not blindly retry the same request across providers. Return a safe result or request appropriate user changes.

21. Do Not Randomly Rotate Providers

Provider switching must not be random. The recommended priority model scores each eligible provider; the provider with the best eligible score receives the request. Randomization may be used only when two providers are intentionally tied.

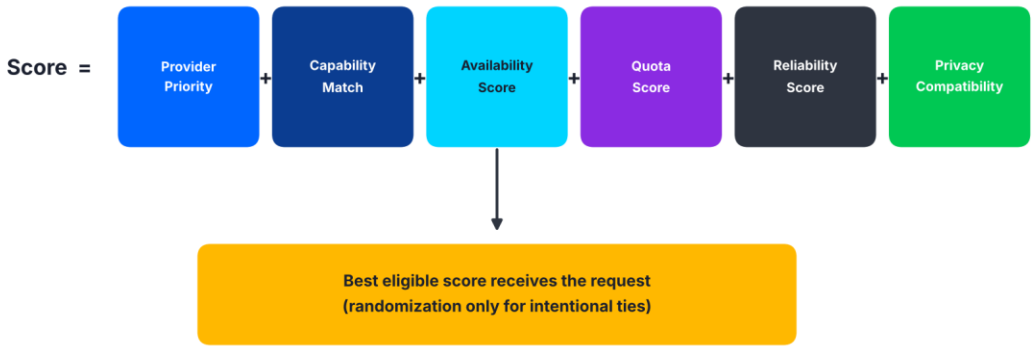


Figure 15 – Provider priority score

22. Provider Cooldown

When a provider reaches a rate limit, CooldownUntil is set to the reset time. The provider router excludes it until the reset time passes or an administrator manually re-enables it. This prevents repeated failures.



Figure 16 – Provider cooldown lifecycle

23. Provider Health Monitoring

The system should maintain:

- Last successful request.
- Last failed request.
- Failure count.
- Rate-limit count.
- Average latency.
- Current cooldown.
- Current availability.
- Model-level health.

Possible States

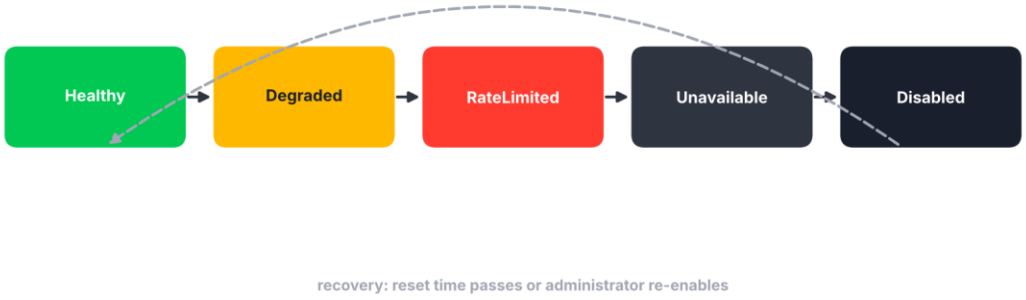


Figure 17 – Provider health states

24. AI Request Caching

Cache can be used for repeated identical or sufficiently identical requests. Candidate cache key components:

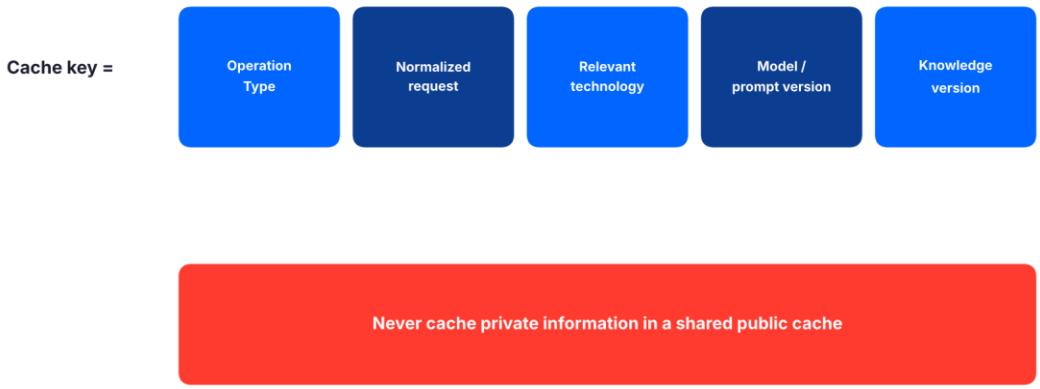


Figure 18 – Candidate cache key components

25. Prompt Management

Prompts must be maintained by operation. Each prompt can have PromptId, Name, Version, Template, IsActive and CreatedAt.



Figure 19 – Prompts maintained per operation

26. Private Code / Data Protection

AI requests may contain private repository content. The platform should classify requests into four sensitivity levels, and provider eligibility depends on policy.

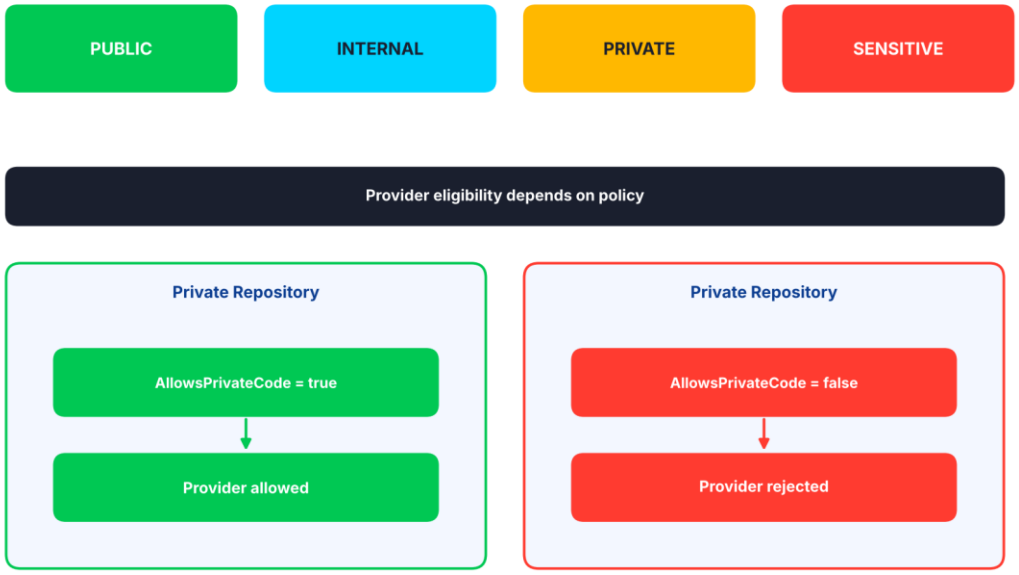


Figure 20 – Request classification and provider eligibility by privacy policy

> **Rule**
> The application must never assume that every free LLM provider has the same privacy policy.

27. GitHub Repository Context Optimization

Do not send an entire repository by default. The preferred pipeline selects the branch, identifies and reads only relevant files, builds a compact context, runs similarity search and then calls the AI Gateway.

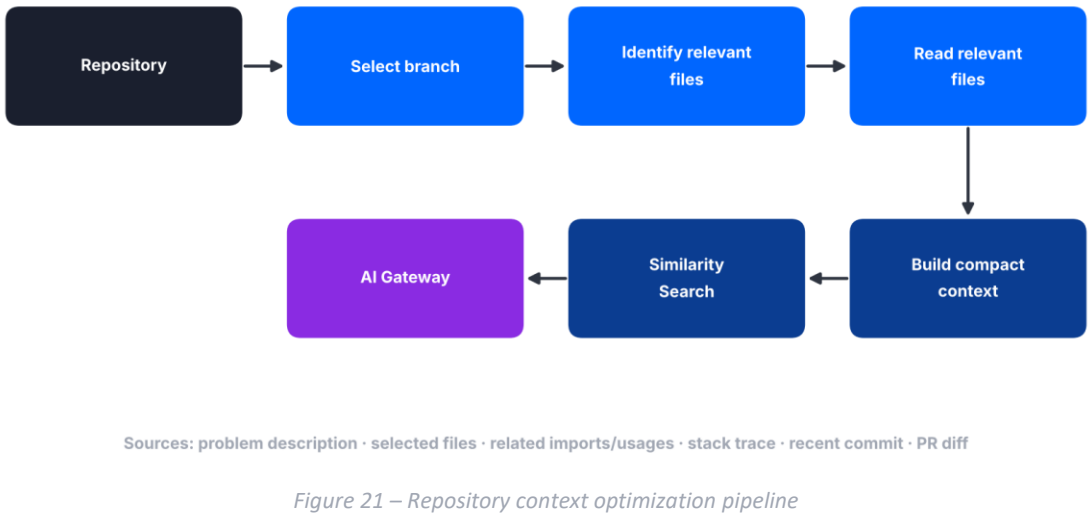


Figure 21 – Repository context optimization pipeline

Possible Context Sources

- Problem description.
- Selected files.
- Related imports/usages.
- Relevant stack trace.
- Recent commit.
- Pull request diff.

28. Image / Screenshot Debugging

When the developer uploads a screenshot:

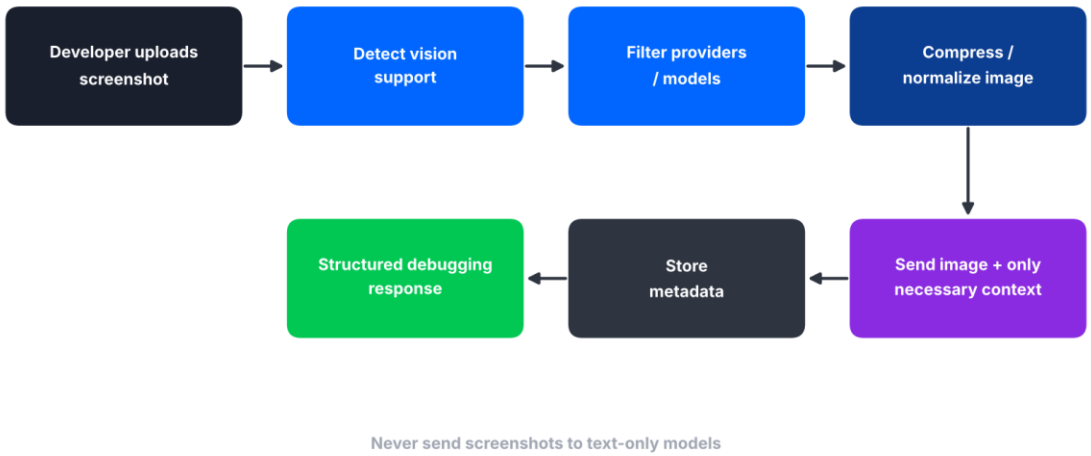


Figure 22 – Screenshot debugging flow

1. Detect whether the selected model supports vision.
2. Filter providers/models.
3. Compress/normalize image when appropriate.
4. Send image plus only necessary technical context.
5. Store metadata.
6. Return structured debugging response.

> **Rule**
> Do not send screenshots to text-only models.

29. Telegram AI Flow

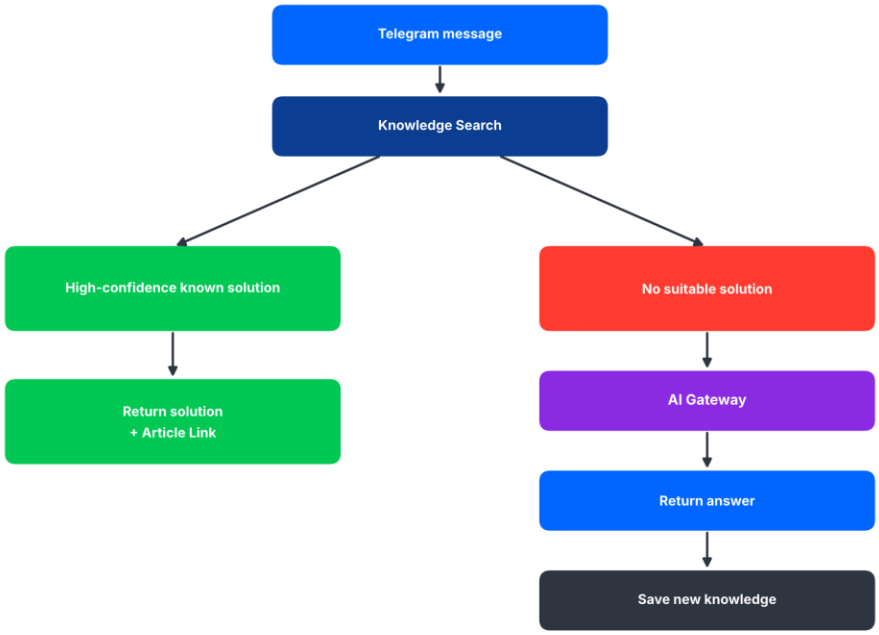


Figure 23 – Telegram AI flow

> **Priority**
> This flow intentionally prioritizes database knowledge before LLM usage.

30. Article Generation Flow

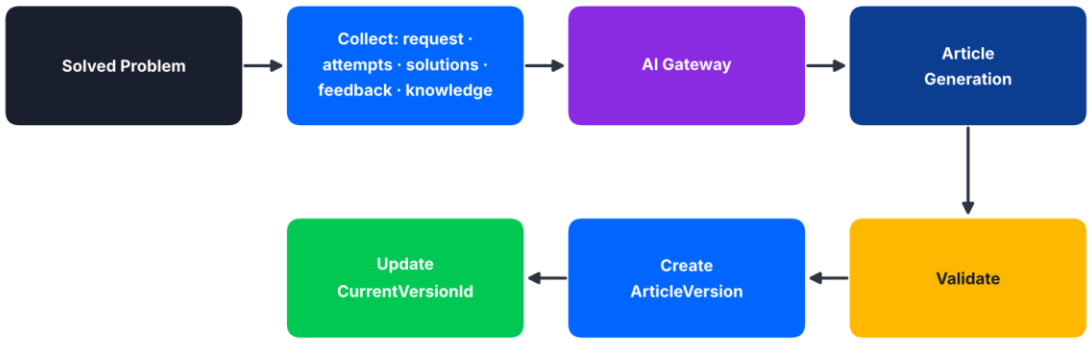


Figure 24 – Article generation flow

31. Challenge Generation Flow

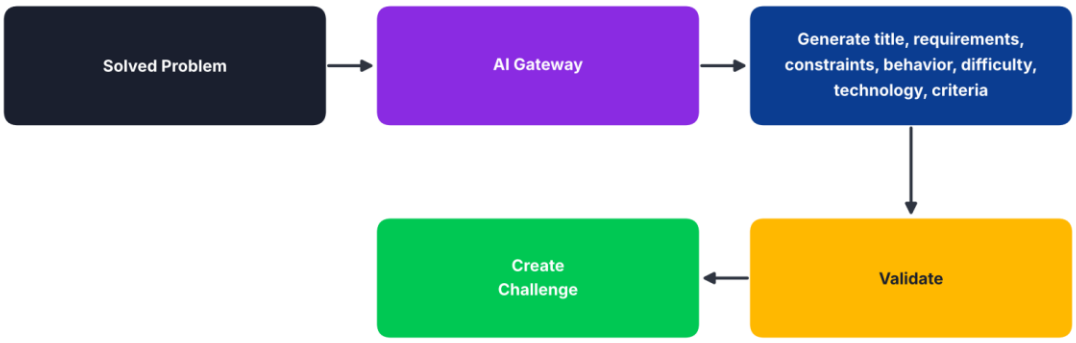


Figure 25 – Challenge generation flow

32. Challenge Evaluation Flow

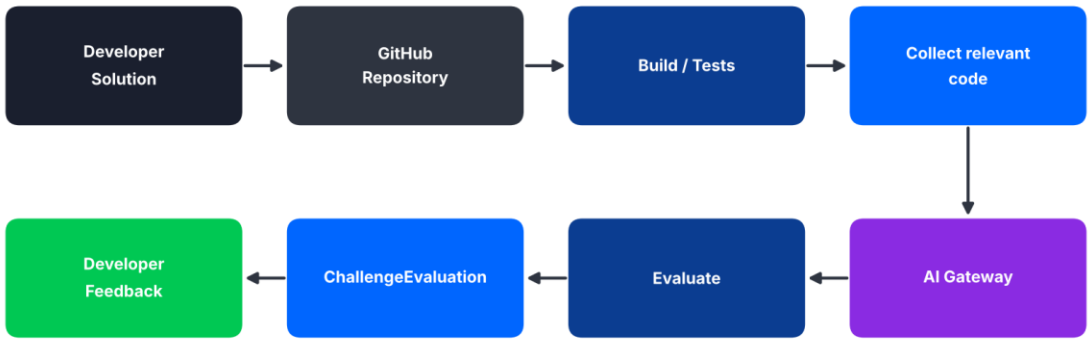


Figure 26 – Challenge evaluation flow

33. Code Review Flow

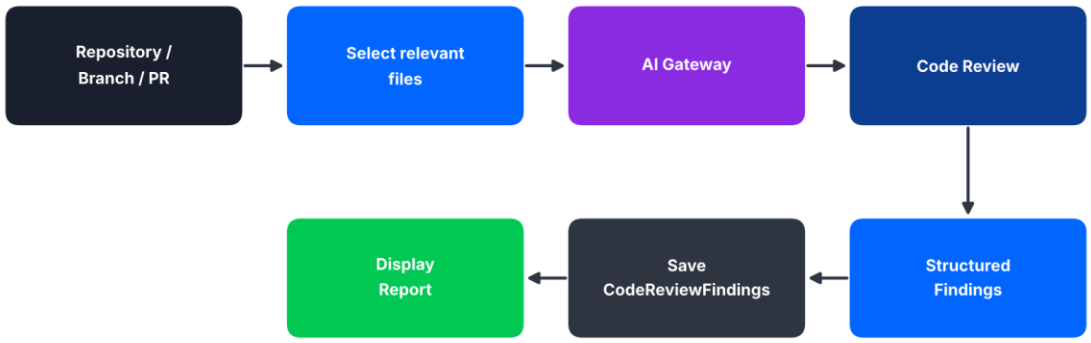


Figure 27 – Code review flow

34. AI Usage Tracking

For each AI request, the following is tracked where available:

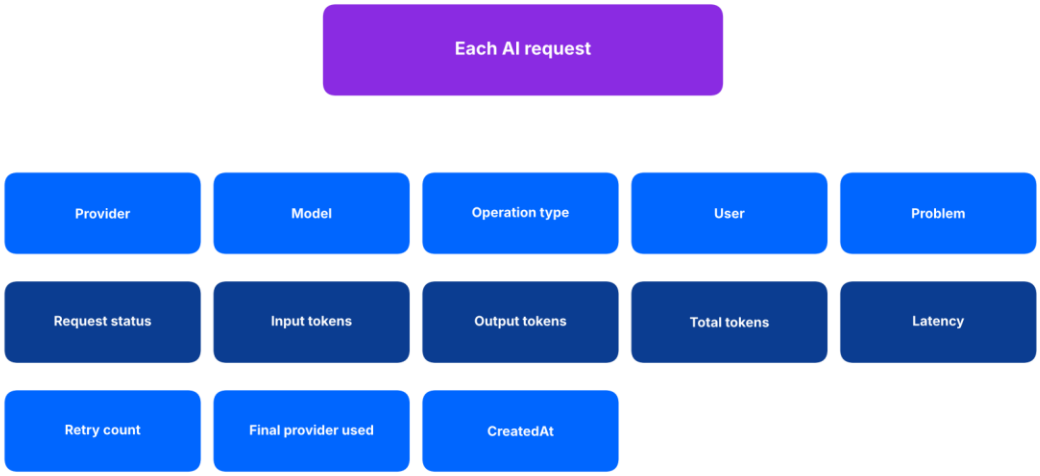


Figure 28 – AI usage tracking fields

This data powers:

- Token optimization.
- Provider comparison.
- Failover.
- Admin analytics.

35. Free-Tier Safety Rule



Figure 29 – What "free" must never be interpreted as

> **Rule**
> The provider catalog and limits must be configurable.

36. Admin AI Dashboard

The Admin area should display four groups of information.

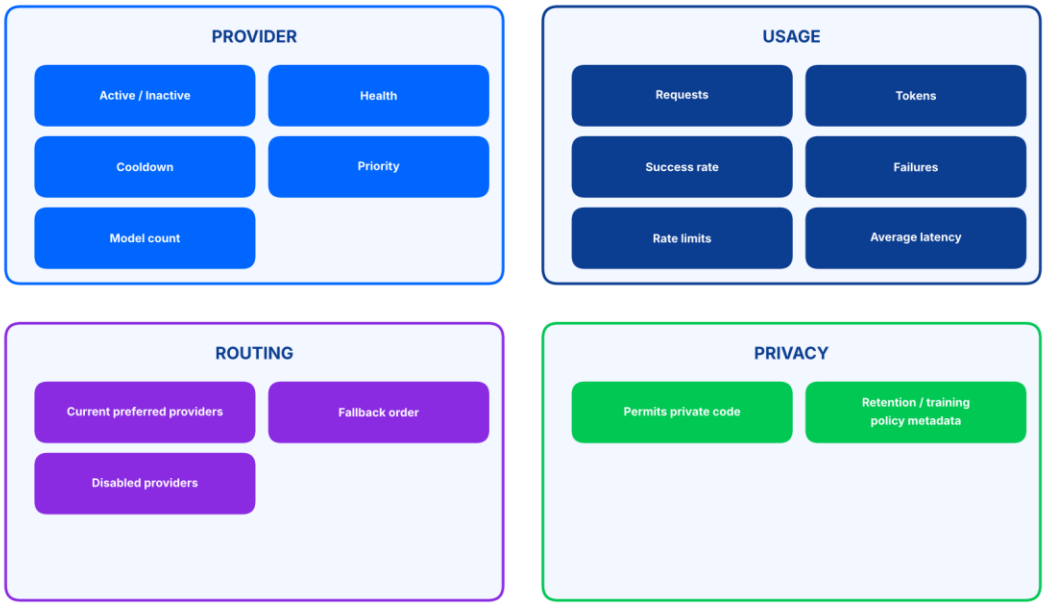


Figure 30 – Admin AI dashboard contents

37. Recommended Initial Routing Strategy

For the MVP, start with a small provider pool instead of integrating every provider immediately.

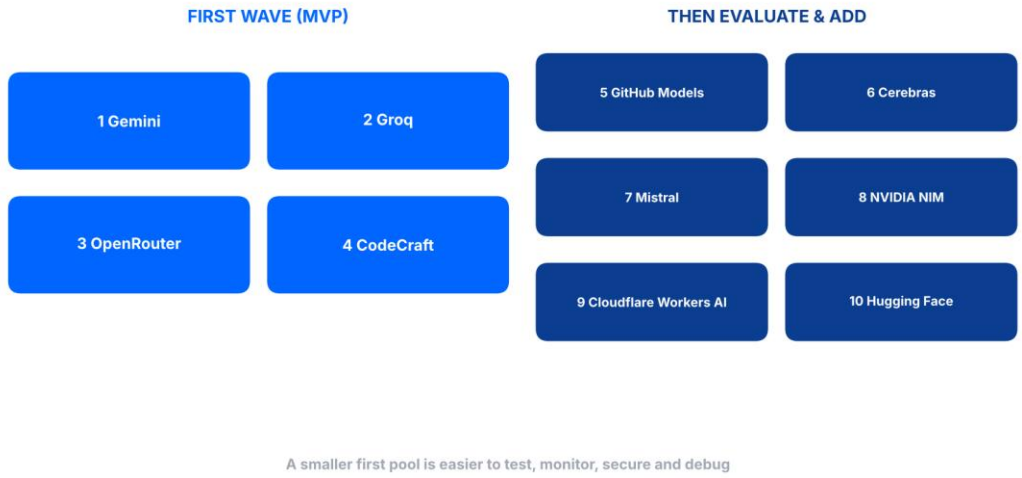


Figure 31 – First wave and follow-up providers

> **Reason**
> A smaller first pool is easier to test, monitor, secure and debug.

38. Development Implementation Order

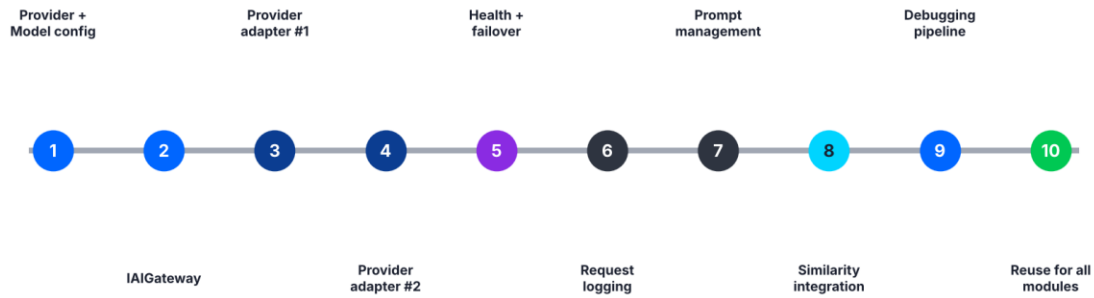


Figure 32 – Ten-phase implementation roadmap

Phase	Scope
1	Create AIProvider + AIModel configuration.
2	Create IAIGateway.
3	Implement one provider adapter.
4	Implement a second provider adapter.
5	Implement provider health and failover.
6	Implement AI request logging.
7	Implement prompt management.
8	Implement similarity search integration.
9	Implement debugging pipeline.
10	Reuse the same AI Gateway for Articles, Challenges, Code Review, Repository Analysis, Challenge Evaluation and Telegram.

39. Final AI Architecture

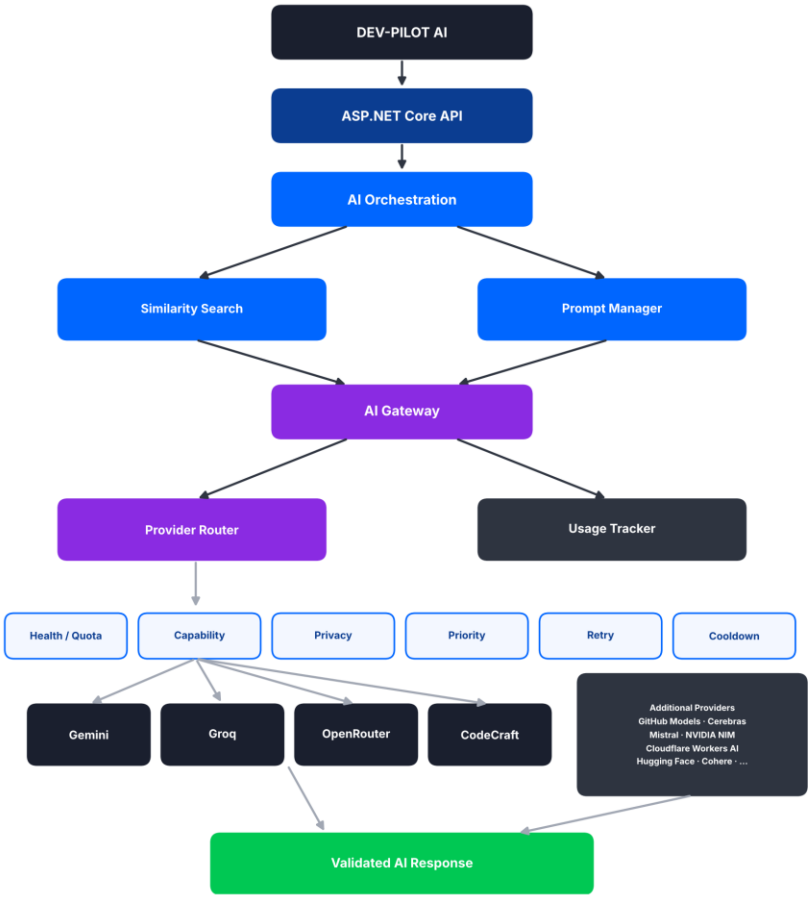


Figure 33 – Final AI architecture

40. Final Design Goal

The AI layer must be:

- Provider-independent.
- Free-tier aware.
- Token-efficient.
- Privacy-aware.
- Fault tolerant.
- Configurable.
- Observable.
- Reusable across all DevPilot AI modules.

> **Key architecture principle**
> Search existing proven knowledge first, use the smallest appropriate AI context second, and use the best available eligible free model through one central AI Gateway.